Which Part of the Loop Helps? A Preregistered Decomposition of Recursive Self-Improvement for Vision-Language-Action Policies

Anonymous CVPR submission

Abstract

Recursive self-improvement (RSI) promises a policy that practises on tasks it writes for itself, keeps only what an external verifier confirms, and revises its own training procedure. We ask whether this promise holds for vision-language-action (VLA) policies, and whether any gain is due to the recursive and meta-level parts of the loop rather than to plain reinforcement learning. We build a framework in which every ingredient is a switch: a verifier gate that accepts or rejects each new version on fresh held-out placements, recursion (practice generated by the latest policy or frozen at the initial one), a verifier (real or permuted success labels), a meta level (none, numeric search, or an LLM that diagnoses round records and edits the training procedure inside a declared space), and a compute-matched one-shot RL baseline. All claims were preregistered with sealed official tests, public amendments, and replicate-averaged reads. On LIBERO-Goal with OpenVLA-OFT (31 runs, four seeds, two GPU platforms) the gated loop never degrades a policy and gains $+1$ to $+2$ points; recursion contributes $+1.7$ points (95% CI $[-0.3, +3.7]$) and the verifier $+1.2$ $[-0.4, +2.9]$, but neither reaches its preregistered bar, and no meta-level variant beats a fixed, hand-tuned procedure (all within ± 1.2 points; the same null on a second, separately registered study). We show why: the object-level loop reaches the same plateau under every procedure in the space, so there is nothing for a meta level to fix, even though the LLM's diagnoses are correct. The meta level only becomes distinguishable from RL when the starting procedure fails; a preregistered self-repair study (sealed, to be read on 2026-11-08) tests exactly that. In a second domain (code world models, four LLMs), the same loop gives verified-feedback gains of $+2.6$ to $+7$ points and recursion is significant at eight rounds ($+2.8$ $[+1.2, +4.5]$ on CRUXEval-O). Finally, we measure that a published VLA-RL checkpoint trained on the benchmark's official initial states is inflated by $+16.8$ points $[+9.8, +23.4]$ relative to fresh placements. We release the framework, every preregistration and all run records.

1. Introduction

A learning system is *recursively self-improving* when the thing being improved is also the thing doing the improving: the current model writes its own practice problems, attempts them, keeps the attempts an external verifier confirms, trains on them, and the improved model writes the next round's problems. Taken one step further, the system also revises the *procedure* by which it improves. Recent language-model results in this direction are striking [2, 5, 22–24], and reinforcement learning with verifiable rewards (RLVR) has become the standard recipe for reasoning models [3, 19]. Robot learning has started to follow: a VLA policy can be improved by RL in simulation from a sparse success signal alone [12, 14].

Two questions are usually left open in such reports. First, *which part of the loop is responsible for the gain?* An RSI loop contains ordinary on-policy RL as a special case; a positive result does not by itself show that recursion, the verifier, or the meta level mattered. Second, *is the gain real?* Benchmarks such as LIBERO [13] have a fixed set of official initial states, and it is tempting to train on them. We found that a widely used VLA-RL checkpoint trained this way is inflated by about 17 points relative to freshly sampled placements of the same tasks (Sec. 8).

This paper treats both questions as experimental design problems. We build a framework (Sec. 3) in which each ingredient of RSI is an independent switch inside one engine: the verifier can be real or permuted, practice can be generated by the latest policy or by the frozen initial one, the meta level can be absent, a numeric search, or a large language model that reads the round records and edits the training procedure, and the whole budget can be spent in one round of plain RL. We then *preregister* every claim (Sec. 4): the comparisons, their bars, the statistics and the read-once rule were committed before the runs, every change is a timestamped public amendment, the official test is sealed until all decisions are made, and finals are averaged over replicate evaluations.

The outcome is mostly negative, and we think that is the useful part. On LIBERO-Goal with an OpenVLA-OFT policy started from one demonstration per task, the gated loop is *safe*—over 31 runs no policy ends below its start beyond

evaluation noise—and it gains one to two points. Recursion is the largest single ingredient (+1.7 points), the verifier the next (+1.2), but neither clears its preregistered bar, and one round of plain RL at the same compute does as well as the loop. Four meta-level variants, including an LLM that may edit nine harness components beyond the usual hyperparameters, are all within ±1.2 points of a fixed procedure, and a second, separately registered meta-level study on both of our platforms gives the same null (−0.3 and +0.3 points). We trace the null to its mechanism: the LLM's diagnoses of the round records are correct and its edits are the ones a practitioner would make, but the object-level loop reaches the same plateau under every procedure in the space, so a meta level has nothing to fix. That suggests where a meta level *can* matter: when the starting procedure fails. We preregistered a self-repair study for exactly that case, starting from a behaviour-cloning recipe known to collapse without the gate; its sealed test is read on 2026-11-08 and the paper will report it as registered (Sec. 6).

In a second domain, code world models [4], the same framework with four LLMs of 1.5–8B parameters gives verified-feedback gains of +2.6 to +7 points over the initial model, and recursion becomes significant only beyond four rounds (+2.8 points at eight rounds, CI excluding zero, replicated on a second model). The contrast with the VLA domain is informative: recursion pays when the frozen initial generator's practice is used up, which did not happen within six rounds on LIBERO-Goal.

Our contributions are: (i) an RSI framework for VLA policies in which every control is a switch inside one engine, with SFT, behaviour cloning and GRPO as interchangeable update operators and an LLM meta level that edits a declared harness space under an attribution guard; (ii) a preregistered, sealed-test protocol for self-improvement claims, with all amendments public; (iii) the first controlled decomposition of an RSI loop on a VLA benchmark, showing safety, small gains, and a null meta level with its mechanism; (iv) a positive recursion result in a second domain at longer horizon; and (v) a measurement of train-on-test inflation in a published VLA-RL result. Code, preregistrations and run records are released.

2. Related work

Self-improvement with verifiers. STaR [23], ReST [5] and ReST-EM [20] fine-tune a language model on its own verified rationales; SPIN [2] and self-rewarding models [22] close the loop with the model as its own judge; Absolute Zero [24] lets the model write its own tasks and uses a code executor as the verifier. RLVR with group-relative policy optimisation [3, 19] is the dominant update rule. These works report that the loop helps; our contribution is to ask which part of the loop helps, with every part switchable and preregistered. The closest analyses are the iterated on-policy convergence results of [11], whose predicted shape (early saturation of a learner restricted to samples from the initial policy) matches our code-domain recursion result.

Meta levels that edit the procedure. LLM-driven search over programs and training recipes has produced strong results when the search space is large and evaluation is cheap [6, 15, 18]. In our setting evaluation is expensive (hundreds of simulator episodes per candidate) and noisy, and the space is a few dozen knobs and harness components. We show that under these conditions an LLM meta level that diagnoses correctly still cannot beat a fixed procedure that already works, and we isolate the case where it could.

RL for VLA policies. OpenVLA [9] and OpenVLA-OFT [10] are the policies we improve; SimpleVLA-RL [12], RL4VLA [14] and related work [16] apply on-policy RL to such policies in simulation with a binary success reward. We use the SimpleVLA-RL training stack as one update operator and its released checkpoints as starting points. Our train-on-test measurement (Sec. 8) concerns the evaluation practice of this line of work rather than its methods.

Preregistration. Preregistered designs are standard in the experimental sciences [17] and have been proposed for machine learning [1]. We follow the practice literally: every bar, statistic and read-once rule was committed to a public repository before the corresponding runs, and every change is a dated amendment that states what had been seen.

3. The framework

Figure 1 shows the three levels of the framework. The *object level* improves a policy; the *gate* decides what is kept; the *meta level* improves the procedure the object level runs. Every control used in our experiments is a flag in the same engine, so a control differs from the arm it is compared with in exactly one place.

3.1. Object level

A *program* π is a small JSON document with four sections: `curriculum` (how many scenes to practise and how to allocate them across tasks), `solver` (samples per scene and sampling temperature), `curation` (which attempts become training items) and `update` (the operator and its hyperparameters). One *inner step* of the loop, run from the current policy θ:

1. **Propose.** Draw fresh scenes. In LIBERO a scene is a freshly sampled object placement for one of the ten tasks (seeds disjoint from every evaluation set); in the code domain the model writes a short program and an input.

2. **Attempt.** The policy tries each scene K times at temperature T.
3. **Verify.** An external oracle the policy cannot change labels each attempt: the simulator's success flag, or a sandboxed interpreter.
4. **Curate.** `mixed_groups` keeps every attempt of scenes whose success rate lies strictly inside a band (used by GRPO); `verified` keeps successful attempts only (used by behaviour cloning, BC).
5. **Update.** A LoRA update of θ with GRPO [19] on the group-centred advantages, or BC/SFT on the kept attempts.

A *round* is three inner steps from the same parent, producing a *candidate* version. The next round starts from the retained version, so practice is always generated by the latest policy: this is the *recursion*.

3.2. Gate, audit and test

Three disjoint evaluation sets are drawn once per run. The *gate* (20 placements per task, 200 episodes, greedy decoding, environment re-seeded per episode so that versions are compared on identical placements) decides: a candidate is accepted iff its gate success is at least the parent's. In the fresh-paired mode used for all main runs a new gate set is drawn every round and the parent is re-evaluated on it, so acceptance is always a paired comparison on placements never seen before. The *audit* (another 200) is recorded but never decides anything; it exposes overfitting to the gate. The *test* is the official benchmark (the 50 official initial states per task, 500 episodes) and is sealed: no process in the loop can read it, and it is read once, after every decision in every run of a study has been made (Sec. 4).

3.3. Meta level

The meta level versions the program π and lets it be improved under the same gate. Each round the incumbent program and $c-1$ proposed programs are each trained for real from the same parent, and the gate picks the version to keep; a program is credited only when its child wins. Three proposers are implemented:

- `fixed`: no change (the meta level is off).
- `numeric`: random mutation of declared knobs, biased toward knobs whose past changes raised the gate.
- `llm`: a language model (Claude Sonnet) reads the current program, the edit space with a neutral description of what each option computes, and, for every past candidate, its program diff, gate score, acceptance and per-step statistics (sampled success, share of mixed scenes, item counts, first/last training loss). It returns a diagnosis and one new program.

The edit space comes in two sizes. The *knob* space has ten hyperparameters (scenes, allocation, samples, temperature, curation rule and budget, operator, learning rate, steps, batch). The *harness* space adds nine components of the training procedure itself: the ratio computation of the policy gradient (legacy or matched), the loss aggregation (token or episode), the BC likelihood temperature (unit or rollout), the advantage normalisation, the curation band, the per-scene cap, the allocation floor and the two clip bounds. Each of these corresponds to a change that, during development, had been made by hand after diagnosing round records; the LLM is never told which option is better.

Attribution guard. The meta level never sees the audit or test, the policy weights, the scenes or the episodes. It only chooses values inside the declared space; every proposal passes the same validator as a numeric mutation; invalid proposals and failed calls fall back to numeric search and are counted. A *compute cap* scales any proposal down so that no candidate does more rollouts or more optimiser work per step than the starting program. All practice data still comes from the policy's own attempts and all verdicts from the oracle, so the LLM is a source of procedure choices, not of data.

3.4. Controls

`nonrec` every round's practice is generated by the *initial* policy (the update still continues from the retained version). Isolates recursion.

`shuffled` success labels are permuted within each scene before curation. Isolates the verifier.

`onebig` one round with the summed rollout and update budget of all rounds. Isolates iteration against plain one-shot RL.

`full`, `llm`, `llm_harness`, `llm_harness_cap` the meta level on, with the numeric proposer, the LLM over the knob space, the LLM over the harness space, and the latter under the compute cap. All have two candidates per round, so `full` and the `llm` arms cost the same.

3.5. Domains

LIBERO / OpenVLA-OFT. The policy is OpenVLA-OFT [10] (7B, action chunks of 8), started from a supervised fine-tune on *one* demonstration per task [12], so that there is room to improve (about 59% on LIBERO-Goal, 19% on LIBERO-10). Rollouts and GRPO/BC training use the SimpleVLA-RL stack with LoRA (r=32). **Code world models.** The model predicts the output of a Python program on an input (the CRUXEval-O format [4]); it writes its own programs, the sandbox executes them, and an attempt is verified iff its answer equals the interpreter's. Qwen2.5-Coder 1.5B and 7B [7], Qwen3-8B [21] and Granite-3.3-8B [8] are trained with LoRA SFT on verified traces.

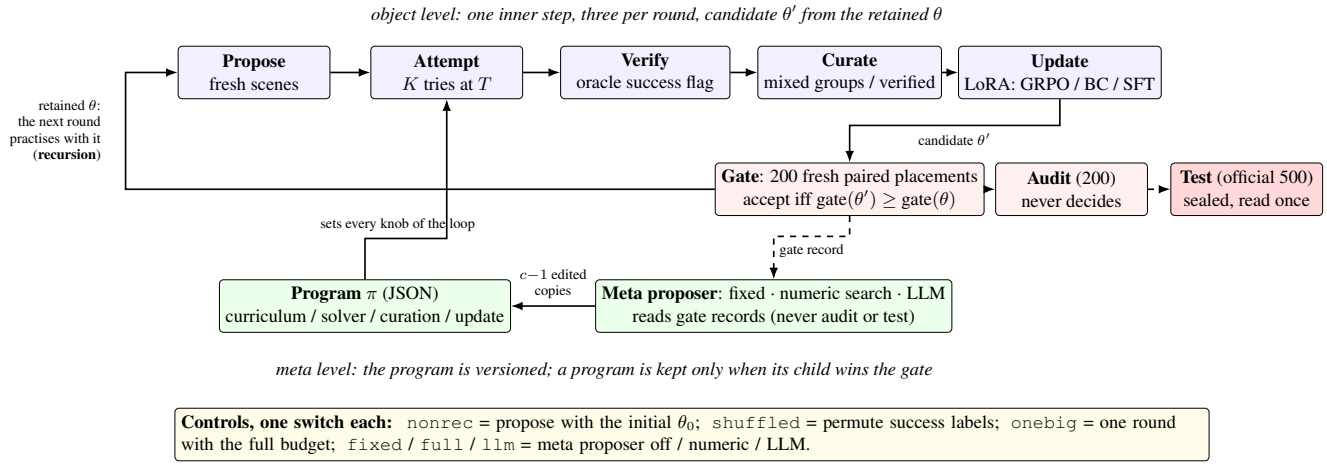


Figure 1. **Framework.** One engine, every ingredient a switch. The object loop practises on fresh scenes with the latest policy; the gate accepts a candidate only if it does not regress on paired held-out placements; the meta level versions the training program and may let a numeric search or an LLM edit it inside a declared space, seeing only gate records. Controls flip one switch each.

4. Preregistered protocol

Self-improvement claims are easy to inflate: the loop itself selects versions, evaluation is noisy, and the person running the study sees intermediate numbers. We therefore fixed the protocol before the runs and treat the preregistration documents (released with the code, 26 dated amendments for the VLA study) as part of the result.

What is sealed. The official LIBERO test states are never used for training, sampling, gating or program selection, and no test file is opened before the study's read-once. Only two sealed pilots per model were allowed before the seed program was frozen in an amendment.

Finals and reference. The evaluation of one 500-episode test is itself noisy: the *same* initial policy evaluated in different runs scored between 57.2% and 60.0% (sd $\approx$ 0.7), and about 40% of episodes flip between nearby policies. Each run's final is therefore the per-episode mean of three evaluations of its retained policy (the run's own test and two replicate evaluations from the same plan and source tree), and the initial reference is the per-episode mean of every in-run evaluation of the initial policy on that platform (36 evaluations on one platform, 21 on the other). A noise check—the same statistic between two halves of the initial-policy evaluations—is reported with every verdict.

Claims and bars (LIBERO-Goal, OpenVLA-OFT). Seed-averaged per-episode paired bootstrap (10,000 draws), 95% CI:

- **R1 improvement**: `fixed` and `full` final $-$ reference $\geq +5$ points, CI excludes 0, and at least 2 of 3 seeds individually above 0.
- **R2 verifier**: `fixed` $-$ `shuffled` $\geq +3$.
- **R3 recursion**: `fixed` $-$ `nonrec` $\geq +2$.
- **R4 meta level**: each meta arm $-$ `fixed` $\geq +2$, reported with the gate–test gap of the selected programs.
- **R5 vs one-shot RL**: the best gated arm (chosen by gate, never by test) $-$ `onebig` $\geq +2$.

A fourth seed was added to R2, R3 and R5 by amendment before any of their runs ended; the 3-seed statistic is reported alongside. Negative readings are reported as such. The reading script was committed before the read and its output is released.

Two platforms. Runs were executed on a shared 8×A800 node and on a 16-GPU cloud pool with its own evaluation stack; the two stacks are not bit-identical, so every comparison is paired within a platform, and the platforms serve as replicates of each other.

5. Decomposing the loop on LIBERO-Goal

Setup. LIBERO-Goal has ten goal-conditioned tasks in one scene. The starting policy is OpenVLA-OFT fine-tuned on one demonstration per task (pooled reference 58.9% on the cloud platform and 58.4% on the A800). The seed program, chosen from sealed pilots and frozen by amendment, is GRPO with learning rate 3×10^{-5}, matched-ratio log-probabilities, episode-balanced loss, 8 samples per scene at temperature 1.5, 160 scenes per inner step, 200 optimiser steps of batch 64 per update; 6 rounds of 3 inner steps; fresh paired gates. All 31 runs (seeds 1–3 for every arm, seed 0 added for the R2/R3/R5 arms) were read once on 2026-11-06 under the rule of Sec. 4.

Table 1. **Registered claims on LIBERO-Goal** (official 500-episode test, 3-evaluation finals, pooled initial reference, seed-averaged paired bootstrap). No bar is met. Platform Y = cloud pool, A = A800.

claim	comparison	points	95% CI	bar	seeds
R1	`fixed` − ref. (Y)	+1.71	[−0.76, +4.23]	+5	1–3
R1	`full` − ref. (Y)	+1.28	[−1.37, +3.99]	+5	1–3
R2	`fixed` − `shuffled` (Y)	+1.22	[−0.37, +2.85]	+3	0–3
R3	`fixed` − `nonrec` (Y)	+1.72	[−0.28, +3.68]	+2	0–3
R4	`full` − `fixed` (Y)	−0.42	[−2.40, +1.56]	+2	1–3
R4	`llm_harness` − `fixed` (Y)	−0.73	[−2.58, +1.07]	+2	1–3
R4	`llm` − `fixed` (A)	+1.04	[−0.84, +3.00]	+2	1–3
R4	`llm_harness_cap` − `fixed` (A)	+1.16	[−0.76, +3.16]	+2	1–3
R5	best gated − `onebig` (Y)	−0.58	[−2.12, +0.95]	+2	0–3
noise	half vs. half of the n0 evaluations (Y / A)	+0.64 / +0.54	[−0.07, +1.36] / [−0.24, +1.35]		

Table 2. **Per-arm finals** (%, mean of the 3-evaluation finals over seeds 1–3; the seed-0 runs are listed separately). Reference: the pooled initial policy, 58.9 (Y) / 58.4 (A).

	`fixed`	`full`	`llm`∗	`nonrec`	`shuffled`	`onebig`
Y, seeds 1–3	60.6	60.2	59.9	59.1	58.9	59.1
A, seeds 1–3	59.1	–	60.1 / 60.2	–	–	–
A, seed 0	57.2	–	–	54.8	57.5	62.7

∗ Y: `llm_harness`; A: `llm` / `llm_harness_cap`.

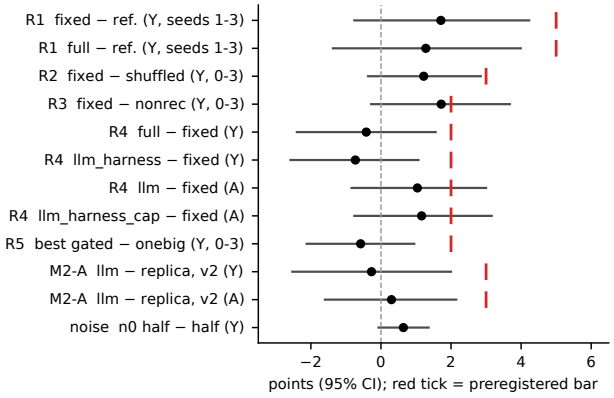


Figure 2. **Registered claims as a forest plot** (points of success rate, 95% CI, red tick = preregistered bar). Every point estimate is positive except the meta-level comparisons, and none reaches its bar; the noise check (bottom) shows the harness itself is stable.

Results. Figure 2 and Tab. 1 give the registered read, Fig. 3 every run, Fig. 4 the gate trajectories, and Tab. 2 the per-arm finals. No preregistered bar is met. The loop's typical gain is one to two points: every gated run ends within noise of or above its start (finals 57.1–62.2 against a 58.4–58.9 reference; the two runs below their reference are within −1.3 and −1.7 points), and the best single seed gains +3.3 [+0.1, +6.6]. Recursion is the largest ingredient (R3, +1.7, CI just including 0; the preliminary single-evaluation read had passed the bar at +2.4 and did not survive replicate averaging), the verifier the next (R2, +1.2); without the verifier the loop ends exactly at its start (57.8–60.7). One round of plain RL with the same budget matches or beats the loop (R5 −0.6; on seed 0 the one-shot policy scores 62.7 against the gated loop's 57.2). All four meta-level arms are within ±1.2 points of `fixed` on both platforms.

What the gate buys. The gate is what makes the loop safe rather than fast. In always-accept pilot curves on LIBERO-10, every recipe stayed within ±3 points of its start for 18 on-policy steps, and the behaviour-cloning recipe of Sec. 6 collapsed from 58.5% to 5.5% on LIBERO-Goal when its versions were accepted unconditionally. Under the gate the same recipes never fall below their start on the sealed test. The price is that a 200-episode paired gate has a standard deviation of about 3.4 points, so it accepts and rejects many candidates on noise; acceptance rates are reported per run in the released records.

LIBERO-10 and LIBERO-Spatial. On LIBERO-10 (long-horizon tasks, 19% start) no sealed pilot recipe reached the preregistered +3 pooled-gate gain within 18 on-policy steps, so by the amendment's rule no main arms were run there and LIBERO-10 is reported as a negative result at this budget. On LIBERO-Spatial a single pilot curve rose from 59.5% to 65.8% (gate+audit, round 4); it was not continued to a sealed read.

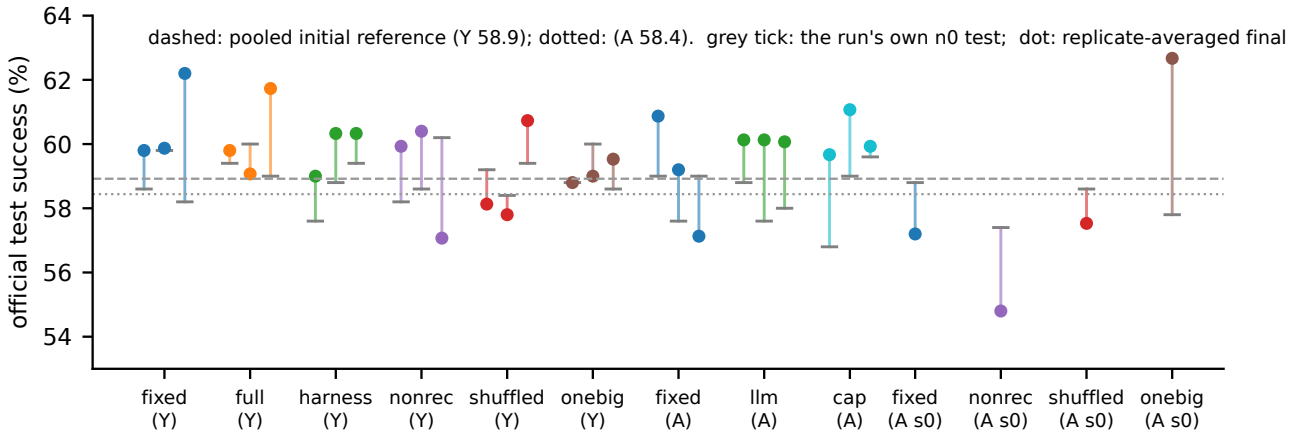


Figure 3. **Every run of the LIBERO-Goal study.** Grey tick: the run's own initial-policy test; dot: the replicate-averaged final of the retained policy; dashed/dotted lines: the pooled initial reference per platform. No gated run ends below its reference beyond noise; the one-shot `onebig` policy of seed 0 is the best single final.

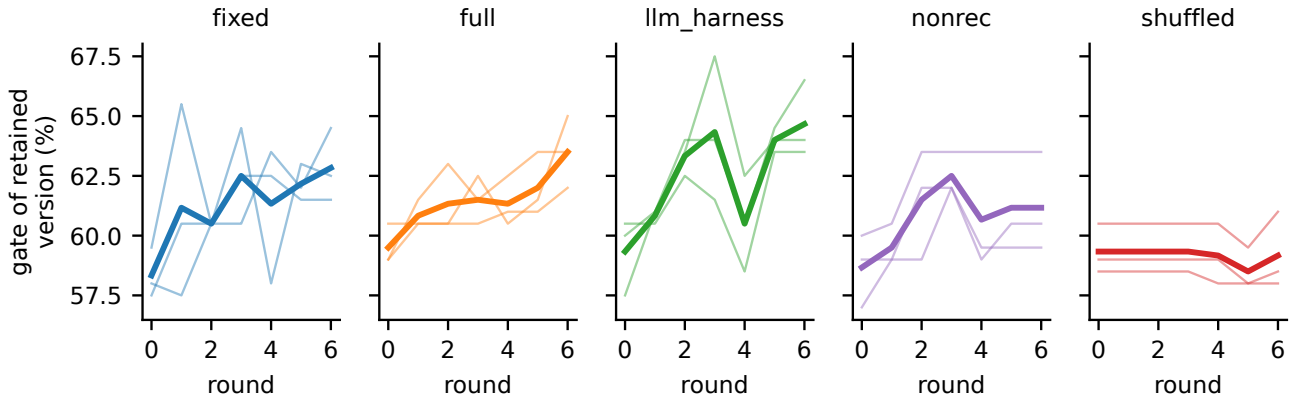


Figure 4. **Gate of the retained version by round** (cloud platform, seeds 1–3 thin, mean bold). The gated arms climb 3–5 points at the gate within six rounds; `shuffled` (verifier removed) stays flat; `nonrec` (practice from the frozen initial policy) stops after round 3. Gate gains overstate test gains by the winner's curse (Tab. 2).

6. When can a meta level matter?

6.1. A second null, and its mechanism

Because the R4 arms compare only each round's newest edit against the incumbent and let the LLM see no per-task signal, we registered a second meta-level study ("v2") in which the LLM sees per-task gate rates with their noise, each candidate starts from its parent's statistics, and the comparison is between an `llm` arm and an identical-program *replica* arm with the same compute, from two starting policies (`pre`: the 1-demonstration start; `tun`: a hand-tuned loop). The primary bar was `llm` − replica $\geq +3$ with CI above 0. It fails on both platforms: -0.27 $[-2.53, +2.00]$ on the cloud pool (seeds 1–3) and $+0.30$ $[-1.60, +2.15]$ on the A800 (seeds 0–3). Pooled over seven seed-pairs, any true effect is within about ± 2 points.

The gate records explain why. Over the v2 study the LLM's proposals scored -1.2 points against the incumbent at the gate (n=30) while identical-program replicas scored $+0.6$ (sd 3.4, n=36): there is no edit effect for a finer gate to resolve. The LLM's diagnoses, logged verbatim, read the loop correctly—it notes the task that is never solved (so GRPO has no signal there), the task that is saturated, and that the remaining gate changes are within noise—and its edits are the ones a practitioner would try (re-weighting those tasks, lowering the learning rate and temperature, z-scoring advantages). None moves success within six rounds. Final retained gates on LIBERO-Goal are 62.8 (`fixed`), 63.5 (`full`), 65.0 (`llm_harness`) and 61.7 (`onebig`), all within gate noise of each other: *the*

object-level loop reaches the same plateau under every procedure in the space, and a correct diagnosis of a procedure that already works has nothing to repair. We therefore did not run a third meta-level study on the tuned start, and report R4 and v2 as the preregistered nulls they are.

6.2. Self-repair: starting from a procedure that fails

A meta level is distinguishable from RL only where RL's procedure fails and a different procedure would not. The study contains one such case, found before any meta-level run: behaviour cloning on the policy's own temperature-1.6 samples with a unit-temperature likelihood *collapses* when its versions are accepted unconditionally (58.5% $\rightarrow$ 5.5% in six rounds on the A800; a milder -6.4 on the cloud platform). The repair is a one-knob change (the BC temperature, the sampling temperature, or the operator), and all three are in the harness space. Shown that collapsing history in a pre-launch call, the LLM diagnosed it correctly and proposed two of the three repairs unprompted. We registered this as a test (amendments 24 and 26) before any of its runs started.

Design. Start program: the collapsing BC recipe. Three arms, four seeds (three on the cloud pool, one on the A800): `fixed` (the gate alone: can it only refuse the collapse?), `full` (numeric search over the harness space, two candidates per round) and `llm_harness` (the LLM over the same space, same compute). Six rounds, fresh paired gates, sealed official test. Bars: **S1** `llm_harness` $-$ `fixed` $\geq +3$, CI excluding 0 (the meta level changes the procedure to effect); **S2** `llm_harness` $-$ `full`, reported with its CI (diagnosis against blind search over the same knobs); **S3** descriptive, against the hand-tuned GRPO loop. Predictions were written down before launch, including the failure case: if S1 fails, the meta level has no demonstrated effect in this study, and the paper says so.

Gate-level record (not a claim). Figure 5 shows the trajectories and Tab. 3 lists what each arm did, from the released round records; the sealed test is read on 2026-11-08 and the registered numbers will replace the placeholders. Three observations hold at the gate level. First, *the gate alone prevents the collapse*: no `fixed` run falls below its start; one seed is held at the initial policy for all six rounds, the other three accept small BC gains (retained gates 60–64.5%). Second, *both proposers find a repair*: numeric search switched one seed to GRPO and another to the temperature-matched BC likelihood, and the LLM lowered the sampling temperature and learning rate on the seed that has finished (retained gate 64.5%; its second seed's accepted LLM programs reach 65.5% at round 3). Third, at the gate all arms end within noise of each other (60–65.5%), so S1 and S2 are decided by the sealed test and not by these records.

A second failing scenario was sought with six gate-only pilots (four extreme GRPO/BC recipes, both platforms); none degraded under the gate, so no second scenario was registered. This is itself a finding about the gate: within the declared space we could not construct a procedure that the gate lets collapse.

7. A second domain: code world models

The same engine, with the sandboxed interpreter as oracle, trains a model to predict program outputs. Programs are written by the model itself (3-shot from a pool that starts with 16 hand-written seeds and grows with admitted programs; a program is admitted only if it is deterministic, literal-valued and not a near-duplicate of any held-out problem), attempted eight times with chain-of-thought, verified by execution, curated to at most two verified traces per case, and used for a LoRA SFT update; the gate is a 400-program validation set proposed by the *initial* model. Held-out tests: CRUXEval-O (primary), CRUXEval-I and LiveCodeBench-execution, read only after retention is frozen. An independent re-scorer agrees with the engine on 99.6–100% of test records.

Table 4 and Fig. 6 summarise the registered comparisons. The verified loop improves every model on at least one test (CRUXEval-O $+2.6$ for the 1.5B model, LCB-execution $+5.8$ for the 7B model, both with CIs excluding 0), the permuted-label control does not, and the difference between them is significant on the trained task family ($+2.7$ $[0.9, 4.5]$). The numeric meta level does not beat the fixed program in any three-seed cell (the one two-seed exception, Qwen3-8B on CRUXEval-I, is not counted as a claim). Recursion is not detectable at four rounds (-0.6) but becomes significant at eight ($+2.8$ $[1.2, 4.5]$, replicated on the 7B model at $+2.9$ $[0.9, 4.8]$ and $+4.5$ $[1.6, 7.4]$ on LCB-execution): the frozen initial generator's practice is as good as the improving model's for about four rounds, after which the non-recursive arm stops accepting gate gains (2 vs 8 accepted gains in rounds 5–8) while the recursive arm keeps finding cases at its new frontier. The extra gain is on the trained task family with no extra forgetting on the secondary suites. This is the shape predicted for iterated on-policy learning [11], and it suggests that the VLA recursion result of Sec. 5 ($+1.7$ at six rounds) is on the same curve before it separates.

8. Train-on-test inflation of a published VLA-RL result

LIBERO is tested on 50 fixed initial states per task. The released SimpleVLA-RL LIBERO-10 checkpoint was trained by RL on exactly those states. We measured how much of

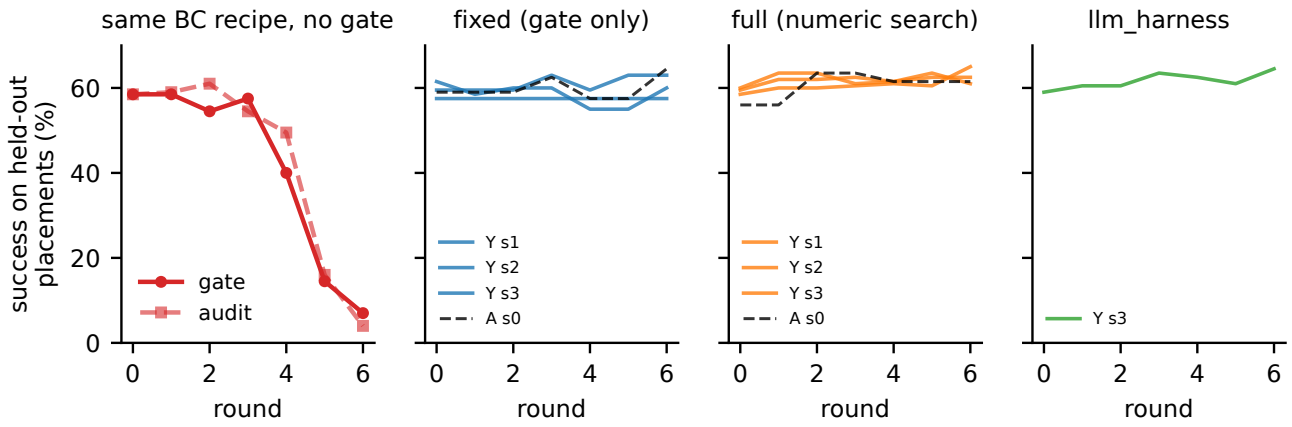


Figure 5. **Self-repair study, gate level.** Left: the starting BC recipe collapses from 58.5% to 5.5% when every version is accepted (A800, gate and audit). Right: under the gate, the same recipe never falls below its start (`fixed`), and both the numeric search and the LLM find repairs that reach 64–65% at the gate; the arms are within gate noise of each other, so the sealed test decides S1 and S2. Runs still in progress are absent.

Table 3. **Self-repair study, gate-level record** (retained version and its gate, %; the program it runs). Sealed test results: [to be inserted after the registered read of 2026-11-08].

seed	`fixed` (gate only)	`full` (numeric search)	`llm_harness`
1 (Y)	r6, 60.0; BC unchanged	r5, 62.5; BC unchanged	*running*
2 (Y)	n0 (all 6 refused)	r6, 65.0; switched to GRPO	*running* (r3: 65.5)
3 (Y)	r5, 63.0; BC unchanged	r6, 61.0; BC, rollout-temp. likelihood	r6, 64.5; BC, T 1.6→1.3, lr 1e-5→2e-5
0 (A)	r6, 64.5; BC unchanged	r4, 64.0 (rounds 5–6 refused)	*running* (r2: 65.5)

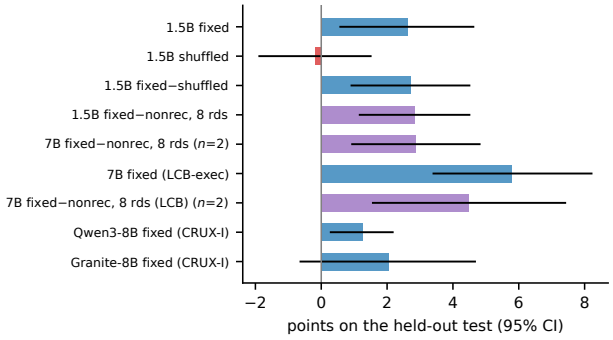


Figure 6. **Code world models.** Verified self-training gains (blue), the permuted-label control (red) and the recursion contrast at eight rounds (purple), with 95% CIs.

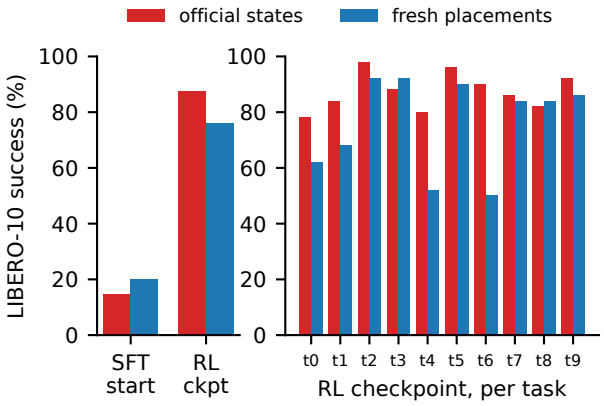


Figure 7. **Train-on-test inflation.** Left: official vs. fresh placements for the SFT start and the RL checkpoint trained on the official states. Right: the RL checkpoint per task; the loss is concentrated on tasks 4 and 6.

its published number this accounts for, as a preregistered secondary measurement: the RL checkpoint and its own SFT starting point were each evaluated greedily on the 500 official episodes and on 500 placements freshly drawn by LIBERO's own sampler (the same distribution, not a perturbation). Inflation is the difference of the two official–fresh gaps, which removes any difficulty difference between the placement sets; the CI is a bootstrap over episodes.

The inflation is $+16.8$ points $[+9.8, +23.4]$ (Tab. 5); a replication on 500 further never-seen placements gives $+16.0$ $[+9.2, +22.4]$, and the loss is concentrated on two of

Table 4. **Code world models**, final − initial on the held-out tests (points, seed-averaged paired bootstrap, 95% CI). Four rounds unless noted; "8 rds" = the long-horizon recursion study. Cells with fewer than three seeds are marked.

model	comparison	CRUXEval-O	CRUXEval-I	LCB-exec
Qwen2.5-C 1.5B	`fixed` − init	$+2.6\ [0.6, 4.6]$	$+1.2\ [-0.8, 3.3]$	$-1.3\ [-4.4, 1.7]$
	`shuffled` − init	$-0.2\ [-1.9, 1.5]$	$-0.1\ [-1.5, 1.3]$	$-2.4\ [-4.7, 0.0]$
	`fixed` − `shuffled`	$+2.7\ [0.9, 4.5]$	$+1.3\ [-0.5, 3.0]$	$+0.9\ [-1.6, 3.5]$
	`full` − `fixed`	$+0.8\ [-0.7, 2.4]$	$-0.8\ [-2.3, 0.6]$	$+0.7\ [-1.7, 3.1]$
	`fixed` − `nonrec`	$-0.6\ [-2.2, 1.1]$	$+0.3\ [-1.3, 1.8]$	$+0.9\ [-1.3, 3.1]$
	`fixed` − `nonrec`, 8 rds	$+2.8\ [1.2, 4.5]$	$+0.2\ [-1.5, 1.9]$	$-0.5\ [-3.1, 2.1]$
Qwen2.5-C 7B	`fixed` − init	$+1.4\ [-0.2, 3.0]$	$+2.0\ [0.2, 3.8]$	$+5.8\ [3.4, 8.2]$
	`full` − `fixed`	$+0.8\ [-0.5, 2.0]$	$+0.1\ [-1.5, 1.6]$	$+1.1\ [-1.0, 3.1]$
	`fixed` − `nonrec`, 8 rds†	$+2.9\ [0.9, 4.8]$	$0.0\ [-2.1, 2.1]$	$+4.5\ [1.6, 7.4]$
Qwen3-8B	`fixed` − init	$+0.5\ [-0.2, 1.3]$	$+1.3\ [0.3, 2.2]$	$+0.2\ [-0.5, 0.9]$
	`full` − `fixed`†	$+0.7\ [-0.8, 2.2]$	$+2.6\ [0.6, 4.7]$	$+0.5\ [-1.0, 2.1]$
Granite-3.3 8B	`fixed` − init	$+1.4\ [-0.8, 3.7]$	$+2.0\ [-0.6, 4.7]$	$+2.4\ [-1.0, 5.9]$

† two seeds.

Table 5. **Train-on-test inflation** (LIBERO-10, success %).

checkpoint	official	fresh	gap
SFT start (never saw the test states)	14.6	20.0	-5.4
RL, trained on the official states	87.4	76.0	$+11.4$
inflation (difference of gaps)			$+16.8\ [9.8, 23.4]$

the ten tasks. The checkpoint is still far better than its start on fresh placements ($+56$ points), so the method works; the point is that about a fifth of the headline number is memorisation of the test states. Every number in this paper is measured on the official states *without* ever training, sampling or selecting on them, which is why our finals should be compared with the "fresh" column of such results rather than the "official" one.

9. Discussion and limitations

What we claim. (1) A gated self-improvement loop for VLA policies is safe at this budget: with fresh paired gates, no run degrades on the sealed test, including runs whose recipe collapses without the gate. (2) Its gains on LIBERO-Goal are small, one to two points, and are not distinguishable from one round of plain RL at equal compute; recursion is the largest ingredient and sits at the edge of its bar. (3) A meta level that edits hyperparameters or harness components does not beat a fixed, tuned procedure, on two studies and two platforms, and the reason is mechanistic: every procedure in the space reaches the same plateau within six rounds. (4) In the code domain the verified loop gains several points and recursion is significant at eight rounds. (5) A published VLA-RL number is inflated by about 17 points by training on the test states.

What we do not claim. We do not claim that recursion or the verifier are useless for VLA policies: the point estimates are positive on every comparison, and the code-domain result shows that recursion separates only after the frozen generator's practice is used up, which six rounds on LIBERO-Goal may not reach. We do not claim that an LLM meta level cannot help; the self-repair study (sealed until 2026-11-08) is the test that can show it, and the paper reports its registered result either way.

Limitations. The VLA results are on one simulated benchmark family with a 7B policy and a few-demonstration start; longer horizons, other policies and real robots are untested here (real-robot data collection is in progress). The gate is noisy (sd 3.4 points at 200 episodes), which both limits the meta level and makes the loop conservative; a larger gate costs proportionally more simulation. Evaluation noise on the official test is about ± 1.5 points per evaluation, which is why finals are replicate-averaged and why bars below $+2$ would not be readable at this budget. Two runs had to be continued after hardware or wall-time interruptions, in each case from a completed round with the same plan; the amendments disclose every such event. Finally, preregistration constrains the analysis but not the design: the seed program and the harness space were chosen by us, and a different space might leave more for a meta level to find.

10. Conclusion

We built a recursive self-improvement framework in which every ingredient is a switch, preregistered every claim with a sealed test, and asked which part of the loop improves a VLA policy. The gate makes the loop safe; recursion and the verifier each contribute about a point and a half; a meta

level that edits the procedure contributes nothing once the procedure works, for a reason we can show in the records; and in a second domain recursion pays only beyond four rounds. We also found that a published VLA-RL number owes about a fifth of itself to the benchmark's fixed test states. We hope the protocol—sealed tests, fresh placements, replicate-averaged finals, public amendments, read-once—is as useful to the community as the results, and that the self-repair study settles, one way or the other, when a meta level is worth its cost.

References

[1] Samuel Albanie, James Thewlis, Sébastien Ehrhardt, and João F. Henriques. On the preregistration of machine learning experiments. *NeurIPS Workshop on Preregistration in ML*, 2021. 2

[2] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. In *ICML*, 2024. 1, 2

[3] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv:2501.12948*, 2025. 1, 2

[4] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. CRUXEval: A benchmark for code reasoning, understanding and execution. In *ICML*, 2024. 2, 3

[5] Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, et al. Reinforced self-training (ReST) for language modeling. *arXiv:2308.08998*, 2023. 1, 2

[6] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *ICLR*, 2025. 2

[7] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2.5-Coder technical report. *arXiv:2409.12186*, 2024. 3

[8] IBM Granite Team. Granite 3.x language models. https://github.com/ibm-granite/granite-3.0-language-models, 2024. 3

[9] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. OpenVLA: An open-source vision-language-action model. In *CoRL*, 2024. 2

[10] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv:2502.19645*, 2025. 2, 3

[11] Kim et al. Coverage improvement and fast convergence of on-policy preference learning. *arXiv:2601.08421*, 2026. 2, 7

[12] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaoyang Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, et al. SimpleVLA-RL: Scaling VLA training via reinforcement learning. *arXiv:2509.09674*, 2025. 1, 2, 3

[13] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks*, 2023. 1

[14] Jijia Liu, Feng Gao, Bingwen Wei, Xinlei Chen, Qingmin Liao, Yi Wu, Chao Yu, and Yu Wang. What can RL bring to VLA generalization? an empirical study. *arXiv:2505.19789*, 2025. 1, 2

[15] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv:2408.06292*, 2024. 2

[16] Guanxing Lu, Wenkai Guo, Chaoyi Zhang, Yue Zhou, Haonan Jiang, Zifeng Gao, Yansong Tang, and Ziwei Wang. VLA-RL: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv:2505.18719*, 2025. 2

[17] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *PNAS*, 115(11):2600–2606, 2018. 2

[18] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv:2506.13131*, 2025. 2

[19] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv:2402.03300*, 2024. 1, 2, 3

[20] Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models. In *TMLR*, 2024. 2

[21] An Yang et al. Qwen3 technical report. *arXiv:2505.09388*, 2025. 3

[22] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. In *ICML*, 2024. 1, 2

[23] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *NeurIPS*, 2022. 2

[24] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv:2505.03335*, 2025. 1, 2